

Temat: Funkcje

Kryteria:

1. Opisuję po co i jak zdefiniować własne funkcje. Podaję zasady nazywania funkcji.
-

PHP –funkcje

Funkcje to wydzielone bloki kodu, przeznaczone do wykonania konkretnego zadania. Dzięki nim unikniemy wielokrotnego powtarzania w skrypcie tych samych instrukcji, a kod stanie się krótszy i bardziej czytelny. W dotychczasowej pracy kilkukrotnie korzystaliśmy z niektórych wbudowanych funkcji, których dostarcza PHP: print, phpinfo, date, count

PHP –funkcje użytkownika

PHP pozwala też definiować **własne funkcje**

Jeżeli powtarzasz w kodzie wielokrotnie tę samą sekwencję instrukcji, warto użyć do tego funkcji.

Wyobraź sobie, że tworzysz program, który wymaga obliczania największego wspólnego dzielnika dwóch liczb wielokrotnie, w różnych miejscach.

Można napisać go raz, dobrze przetestować i stworzyć z niego własną funkcję.

PHP –funkcje użytkownika

Definicja funkcji musi nastąpić przed jej wywołaniem i ma postać:

```
function nazwa_funkcji()  
{  
    //instrukcje wnętrza funkcji  
}
```

Raz utworzoną funkcję można następnie wywoływać dowolną liczbę razy w kodzie skryptu.

Wywołanie polega na podaniu nazwy zakończonej nawiasem okrągłym i powoduje wykonanie instrukcji znajdujących się wewnątrz funkcji.

PHP – funkcje użytkownika

```
<h3>Funkcja1:</h3>
<?php
    function func1(){
        echo "Instrukcje funkcji zostały wykonane... ";
    }
    func1();
?>
```

Funkcja1:

Instrukcje funkcji zostały wykonane...

PHP –funkcje użytkownika

- ✓ W nazywaniu funkcji obowiązują podobne zasady jak przy zmiennych. Nazwa może się składać z liter, cyfr oraz znaków podkreślenia, nie może się jednak zaczynać od cyfry.
- ✓ Tworzona funkcja nie powinna nadpisywać funkcji wbudowanych, nie powinna się więc tak samo nazywać jak funkcje wbudowane w php.
- ✓ Warto by nazwy funkcji dobrze opisywały jej przeznaczenie

Kryteria:

1. Opisuję po co i jak zdefiniować własne funkcje. Podaję zasady nazywania funkcji.

Pokaż światłami/kciukami jak oceniasz, na ile spełniasz kryterium

Kryteria:

2. Opisuję jak zdefiniować funkcje z argumentami oraz w jaki sposób ustawić wartość domyślną argumentu.

PHP –Argumenty funkcji

- ✓ Funkcjom można przekazywać argumenty/parametry, które mogą wpływać na działanie funkcji bądź też mogą być przez funkcje przetwarzane.
- ✓ Nie są konieczne, jednak większość funkcji z nich korzysta.
- ✓ Argumenty, to wartości potrzebne do wykonania zadania.
- ✓ Listę argumentów (oddzielonych od siebie znakami przecinka) należy umieścić w nawiasie okrągłym.
- ✓ Jeżeli funkcja zdefiniowana jest z argumentami, podczas wywołania konieczne jest ich podanie.

PHP –Argumenty funkcji

Schematycznie taka konstrukcja ma następującą postać:

```
function nazwa_funkcji(argument1, argument2,  
... , argumentN)  
{  
    //instrukcje wnętrza funkcji  
}
```

PHP – Argumenty funkcji

```
<h3>Funkcja przyjmująca 2 argumenty</h3>
<?php
    function printName($fname,$sign)
    {
        echo $fname . " Nowak" . $sign . "<br>";
    }
    echo "Nazywam się ";
    printName("Adam",".");
    echo "Moja siostra nazywa się ";
    printName("Magda","!");
    echo "Mój brat nazywa się ";
    printName("Jan","?");
?>
```

Funkcja przyjmująca 2 argumenty

Nazywam się Adam Nowak.
Moja siostra nazywa się Magda Nowak!
Mój brat nazywa się Jan Nowak?

PHP – Argumenty funkcji

<h3>Funkcja przyjmująca argument - wykorzystanie danych z tablicy:</h3>

```
<?php
function dajImie($imie_f){
    //funkcja pobierająca jeden argument
    echo 'Mam na imie '.$imie_f.', miło was poznać';
}
$imiona = array('Paweł', 'Piotr', 'Stanisław', 'Dawid');
foreach($imiona as $imie){
    dajImie($imie);
    echo "<br>";
    // $imie przesyłane do funkcji, w funkcji dostępne jako $imie_f
}
?>
```

Funkcja przyjmująca argument - wykorzystanie danych z tablicy:

Mam na imie Paweł, miło was poznać
Mam na imie Piotr, miło was poznać
Mam na imie Stanisław, miło was poznać
Mam na imie Dawid, miło was poznać

PHP –Wartość domyślna argumentu

- ✓ Argumenty mogą przyjmować **wartością domyślną**, podaną przy definicji funkcji (w liście argumentów) .
- ✓ Jeśli podczas wywołania funkcji nie podamy wartości argumentu, przyjmie on wartość domyślną.
- ✓ Argumentów domyślnych może być więcej, **umieszczamy je zawsze na końcu listy argumentów.**
- ✓ Wartość domyślna musi być wyrażeniem statycznym, a nie (na przykład) zmienną, elementem klasy lub wywołaniem funkcji.

PHP – Wartość domyślna argumentu

<h3>Wartość domyślna argumentu</h3>

```
<?php
    function cityCountry($city, $country='Polska')
    {
        echo $city.' znajduje się w państwie '.$country.'<br>';
    }
// wywołanie funkcji z jednym argumentem
cityCountry('Warszawa');
// wywołanie funkcji z 2 argumentami
cityCountry('Tirana', 'Albania');
?>
```

Wartość domyślna argumentu

Warszawa znajduje się w państwie Polska
Tirana znajduje się w państwie Albania

Kryteria:

2. Opisuję jak zdefiniować funkcje z argumentami oraz w jaki sposób ustawić wartość domyślną argumentu.

Pokaż światłami/kciukami jak oceniasz, na ile spełniasz kryterium

Kryteria:

3. Przedstawiam działanie instrukcji return i jej znaczenie dla poprawnego działania funkcji.
-

PHP – Zwracanie wartości

- ✓ Funkcje oprócz przyjmowania argumentów mogą również zwracać różne wartości, np. wyniki wykonanych przez siebie obliczeń.
- ✓ Do zwracania wartości służy instrukcja **return**.
- ✓ Wykonanie tej instrukcji powoduje jednocześnie przerwanie wykonywania funkcji.
- ✓ Wartość przeznaczoną do zwrócenia należy umieścić za instrukcją **return**.
- ✓ Instrukcji **return** można również użyć bez żadnego parametru. Powoduje ona wtedy jedynie przerwanie działania funkcji.

PHP – Zwracanie wartości

Schematycznie konstrukcja ma następującą postać:

```
function nazwa_funkcji()  
{  
    //instrukcje ciała funkcji  
    return wartość;  
}
```

PHP – Zwracanie wartości

<h3>Zwracanie wartości przez funkcję</h3>

```
<?php
```

```
function calculateDiscount($price, $discount=10)
```

```
{
```

```
    $disPrice=$price-(($price*$discount)/100);
```

```
    return $disPrice;
```

```
    /* return zwraca wartość zmiennej $disPrice,  
    nie wypisuje jej jednak*/
```

```
}
```

```
// można wypisać przy pomocy echo lub print
```

```
print calculateDiscount(100,20);
```

```
// lub zapamiętać w zmiennej do dalszego wykorzystania
```

```
$abc= calculateDiscount(100);
```

```
$abcd=$abc+300;
```

```
echo "<br>".$abcd;
```

```
?>
```

Zwracanie wartości przez funkcję

80

390

PHP – Zwracanie wartości

Wartością zwracaną może być tablica, czy obiekt.

```
<h3>Funkcja tworząca i wyświetlająca tablicę</h3>
```

```
<?php
```

```
function holidays($a, $b, $c){  
    $holidaysT=array($a, $b, $c);  
    echo "<pre>";  
    print_r ($holidaysT);  
    echo "</pre>";  
    return $holidaysT;  
}
```

```
holidays("Boże Narodzenie", "Nowy Rok", "Trzech króli");  
holidays("1 Maja", "3 Maja", "Koniec roku szkolnego");
```

```
?>
```

Funkcja tworząca i wyświetlająca tablicę

```
Array  
(  
    [0] => Boże Narodzenie  
    [1] => Nowy Rok  
    [2] => Trzech króli  
)  
  
Array  
(  
    [0] => 1 Maja  
    [1] => 3 Maja  
    [2] => Koniec roku szkolnego  
)
```

Kryteria:

3. Wyjaśniam działanie instrukcji return i jej znaczenie dla poprawnego działania funkcji.

Pokaż światłami/kciukami jak oceniasz, na ile spełniasz kryterium

Kryteria:

4. Wyjaśniam znaczenie zasięgu zmiennych dla poprawnego działania funkcji. Opisuję zmienne lokalne, globalne i statyczne.

PHP – Zasięg zmiennych

- ✓ Korzystając z funkcji, należy zwrócić uwagę na kwestię zasięgu zmiennych.
- ✓ Zasięg danej zmiennej to inaczej miejsca w skrypcie, w których jest ona ważna.
- ✓ Zmienne w PHP mogą być **lokalne, globalne i superglobalne**.
- ✓ **zmienne globalne są dostępne w całym skrypcie z wyjątkiem wnętrza funkcji.**
- ✓ **W funkcjach** zdefiniowanych przez użytkownika zmienne mają **zasięg lokalny**.

PHP – Zasięg zmiennych

<h3>Zasięg zmiennej</h3>

<?php

```
$a = 7; // zasięg globalny
function writeA(){
    echo 'zmienna $a ma wartość: '.$a.'<br>';
    // odwołanie do zmiennej o zasięgu lokalnym
}
writeA(); // wywołanie funkcji
echo "Wartość a = $a <br>";
// wypisanie wartości zmiennej bez funkcji
```

?>

Zasięg zmiennej

Notice: Undefined variable: a in C:\xampp\htdocs\ZSK\l6p7.php on line 12
zmienna \$a ma wartość:
Wartość a = 7

PHP – Zasięg zmiennych

Aby mieć możliwość dostępu do zmiennych globalnych wewnątrz funkcji można zastosować dwa sposoby:

- ✓ pierwszy to odwołanie się do zmiennej **superglobalnej** **\$GLOBALS**,
- ✓ drugi to użycie słowa kluczowego **global** (**niezalecane w php7**).

PHP – instrukcja global

Użycie wewnątrz funkcji instrukcji global w ogólnej postaci:

`global nazwa_zmiennej;`

umożliwia wykorzystanie zmiennej globalnej wewnątrz funkcji.

Jeśli istnieje więcej zmiennych globalnych, które mają być użyte wewnątrz funkcji, można użyć konstrukcji:

`global zmienna1, zmienna2, zmienna3;`

Inaczej ma się sprawa ze stałymi. Te są dostępne zawsze

PHP – instrukcja global

```
<h3>Zasięg zmiennej</h3>
<?php
    $a = 7; // zasięg globalny
    define('A', 3.14);
    function writeA(){
        global $a;
        //jeżeli zmienna $a jest zadeklarowana,
        //to jest widoczna w ciele funkcji
        echo 'zmienna $a ma wartość '.$a.'<br>dostaliśmy
        się do niej dzięki znacznikowi global<br>';
        echo 'stała A ma wartość
        '.A.'<br>widzimy ją bez sztuczek';
    }
    writeA(); // działa tak jak chcieliśmy
?>
```

Zasięg zmiennej

zmienna \$a ma wartość 7
dostaliśmy się do niej dzięki znacznikowi global
stała A ma wartość 3.14
widzimy ją bez sztuczek

PHP – instrukcja global

- ✓ Słowo kluczowe **global** informuje PHP, że wymienione po nim zmienne mają zostać zaimportowane ze stosu głównego. Działa ono nawet wtedy, jeśli zmienna o danej nazwie nie istnieje, dlatego korzystanie z funkcji używających **global** musi być bardzo uważne.
- ✓ **Uwaga!** Używanie **global** w funkcjach jest obecnie uznawane za złą praktykę programistyczną z powodu wielu problemów z testowaniem i przewidywalnością działania tak napisanych programów.

PHP – \$GLOBALS

Korzystanie ze zmiennej superglobalnej — \$GLOBALS.

Zmienne superglobalne to takie, które dostępne są zawsze w każdej części skryptu. (o innych zmiennych tego typu na kolejnych lekcjach).

\$GLOBALS to tablica zawierająca odniesienie do każdej zmiennej zdefiniowanej przez użytkownika, która ma zasięg globalny dla danego skryptu .

Kluczami tej tablicy są nazwy, a wartościami kluczy — wartości zmiennych.

Aby z niej skorzystać należy zastosować konstrukcję:

`$GLOBALS["nazwa_zmiennnej"]`

PHP – \$GLOBALS

<h3>Zasięg zmiennej</h3>

```
<?php
    $s = 25;
    $t = 50;
    function subtraction()
    {
        $GLOBALS['v'] = $GLOBALS['t'] - $GLOBALS['s'];
    }
    subtraction();
    echo $v;
?>
```

Zasięg zmiennej

25

v jest zmienną występującą w tablicy \$GLOBALS, jest ona dostępna również poza funkcją

PHP – Zmienne lokalne funkcji

- ✓ Jak sama nazwa wskazuje, są one przeciwieństwem zmiennych globalnych, a zatem **są dostępne jedynie wewnątrz funkcji, w której zostały zdefiniowane.**
- ✓ Zmienne inicjowane wewnątrz funkcji są niszczone z chwilą kończenia funkcji
- ✓ Bardzo łatwo się o tym przekonać, odwołując się do takiej zmiennej w innej części skryptu.

PHP – Zmienne lokalne funkcji

```
<h3>Błędne odwołanie do zmiennej lokalnej</h3>
<?php
    function func()
    {
        $number1 = 100;
        echo "Wartość zmiennej \$number1 = $number1 <br>";
    }
    func();
    echo "Wartość zmiennej \$number1 = $number1 <br>";
?>
```

Błędne odwołanie do zmiennej lokalnej

Wartość zmiennej \$number1 = 100

Notice: Undefined variable: number1 in C:\xampp\htdocs\ZSK\l6p10.php on line 16
Wartość zmiennej \$number1 =

PHP - Zmienne statyczne

- ✓ **Zmienne statyczne** -to zmienne lokalne funkcji, które zachowują swoją wartość między wywołaniami funkcji.
- ✓ Zwykła zmienna lokalna jest widoczna tylko w obrębie funkcji i jest tworzona tylko w momencie wywołania tej funkcji. Gdy funkcja zakończy działanie, wszystkie jej zmienne tracą swoje wartości.
- ✓ Zmienna statyczna istnieje tylko w zasięgu funkcji lokalnej, ale nie traci swojej wartości, gdy program opuszcza ten zakres.

PHP - Zmienne statyczne

- ✓ Jeżeli w ciele funkcji deklarujemy jakieś zmienne, to za każdym razem gdy wywołamy funkcję będą one inicjowane na nowo.
- ✓ Gdy chcemy, by zmienna inicjowana była podczas pierwszego wywołania funkcji a następnie zapamiętywana do kolejnego wywołania musimy użyć w deklaracji słowa **static** .
- ✓ Od tej chwili, za każdym razem, gdy wywoływana jest funkcja, ta zmienna będzie nadal zawierała informacje, które zawierała od ostatniego wywołania funkcji.

PHP - Zmienne statyczne

✓ Jeśli przy tworzeniu zmiennej zostanie użyte słowo **static**, schematycznie:

`static $nazwa_zmiennej = $wartość;`

wtedy wartość zmiennej będzie przechowywana między wywołaniami funkcji.

✓ pierwsze wywołanie funkcji spowoduje utworzenie zmiennej statycznej i zapamiętanie jej ostatecznej wartości. W każdym kolejnym wywołaniu instrukcja przypisująca pierwotną wartość zmiennej będzie ignorowana, a używana będzie wartość z poprzedniego wywołania.

PHP – Zmienne statyczne

<h3>Zapamiętywanie zmiennej do następnego wywołania </h3>

<?php

```
function ileWywolan(){  
    static $ilosc = 0; //deklaracja zmiennej static  
    return ++$ilosc;  
}
```

```
function bezStatic(){  
    $ilosc = 0; //deklaracja zmiennej bez static  
    return ++$ilosc;  
}
```

```
echo '<h3> wywołania funkcji ze static</h3>';  
echo 'ilosc wywolan '.ileWywolan().'<br>';  
echo 'ilosc wywolan '.ileWywolan().'<br>';  
echo 'ilosc wywolan '.ileWywolan().'<br>';  
echo '<h3> wywołania funkcji bez static</h3>';  
echo 'ilosc wywolan '.bezStatic().'<br>';  
echo 'ilosc wywolan '.bezStatic().'<br>';  
echo 'ilosc wywolan '.bezStatic();
```

?>

wywołania funkcji ze static

ilosc wywolan 1
ilosc wywolan 2
ilosc wywolan 3

wywołania funkcji bez static

ilosc wywolan 1
ilosc wywolan 1
ilosc wywolan 1

PHP – Zmienne statyczne

Jaki będzie efekt działania skryptu?:

```
<?php
function colour()
{
    static $i = 0;
    $i++;
    if($i % 2 == 0)
    {
        return '#ffffff';
    }
    return '#cccccc';
}
echo '<table style="width:10%; text-align: center;">';
for($x = 0; $x < 10; $x++)
{
    echo '<tr><td style="border: 1px solid black;
    background-color:'.colour().'">'.$x.'</td></tr>';
}
echo '</table>';
?>
```

<http://localhost/zsk/l6p12.php>

Kryteria:

4. Wyjaśniam znaczenie zasięgu zmiennych dla poprawnego działania funkcji. Opisuję zmienne lokalne, globalne i statyczne.

Pokaż światłami/kciukami jak oceniasz, na ile spełniasz kryterium

Kryteria:

5. Opisuję sposoby przekazywania argumentów

PHP – sposoby przekazywania argumentów

Argumenty funkcji mogą być przekazywane na dwa różne sposoby:

- ✓ **Przekazywanie przez wartość**
- ✓ **Przesyłanie przez referencję**

PHP – przekazywanie przez wartość

- ✓ W dotychczasowych przykładach było to tzw. **przekazywanie przez wartość**
- ✓ Oznacza to, że funkcja tak naprawdę otrzymuje kopię argumentu i wszelkie operacje wykonuje na tej kopii. Stąd nie może się zmienić stan zmiennej poza funkcją.
- ✓ Żadna zmiana nie narusza oryginału.

PHP – przekazywanie przez wartość

```
<?php
    $x=10;
    function divideNumber($number){
        return $number/=2;
    }
    echo divideNumber($x). '<br>'; //modyfikacja
    echo $x; // wartość zmiennej nie ulega zmianie
?>
```

5

10

PHP – przekazywanie przez referencję

Jeżeli chcielibyśmy modyfikować dany argument tak, by zmiana w funkcji pociągała za sobą zmianę w oryginale musimy **przesłać go przez referencję**.

Aby to osiągnąć, należy przed nazwą parametru dodać znak **&** (*ang. **ampersand***).

Nie ma znaku odwoływania się przez referencję przy wywoływaniu funkcji - istnieje tylko w definicji funkcji.

PHP – przekazywanie przez referencję

Przypisanie zmiennych przez referencję - oznacza, że nowa zmienna odnosi się (innymi słowy, "staje się aliasem" lub "wskazuje na") do pierwotnej zmiennej.

Zmiany wykonane na nowej zmiennej oddziałują także na pierwotną zmienną i vice versa.

Referencje pozwalają na stworzenie dwu zmiennych zawierających tę samą zawartość.

Zapis: **`$a =&$b`** znaczy tyle, że `$a` oraz `$b` wskazują na tę samą zmienną. `$a` oraz `$b` są całkowicie równe, czyli nie `$a` wskazuje na `$b` lub odwrotnie, lecz `$a` oraz `$b` wskazują na to samo miejsce.

PHP – przekazywanie przez referencję

```
<h3>Przekazywanie zmiennej przez referencję </h3>
<?php
    $x=10;
    function divideNumber(&$number) {
        // dodajemy znak & przed zmienną w argumencie
        return $number/=2;
    }
    echo 'wartość początkowa ' . $x . '<br>';
    echo 'działanie funkcji: ' . divideNumber($x) . '<br>';
    echo 'wartość zmiennej po funkcji ' . $x;
?>
```

Przekazywanie zmiennej przez referencję

wartość początkowa 10

działanie funkcji: 5

wartość zmiennej po funkcji 5

Kryteria:

5. Opisuję sposoby przekazywania argumentów

Pokaż światłami/kciukami jak oceniasz, na ile spełniasz kryterium

Kryteria:

6. Opisuję sposoby deklaracji typu argumentów i wartości zwracanych przez funkcję

PHP – argumenty i typy danych

Deklaracje typów pozwalają funkcjom wymagać, aby parametry były określonego typu.

Jeśli podana wartość jest niepoprawnego typu, generowany jest błąd (w trybie ścisłym) lub następuje automatyczna konwersja typów.

php jest elastyczne w stosowaniu typów. Często następuje automatyczna, niejawna konwersja typów.

Ścisłe kontrolowanie typów danych w niektórych sytuacjach jest jednak niezbędne.

PHP – argumenty i typy danych

Począwszy od PHP7 dopuszczono deklarowanie typów prostych (bool, float, int, string).

Aby wskazać, że dany argument ma mieć konkretny typ, należy nazwę argumentu poprzedzić nazwą typu — schematycznie:

```
function nazwa_funkcji(typ1 argument1, typ2  
argument2, ..., typN argumentN)  
{  
// Instrukcje z wnętrza funkcji  
}
```

PHP – argumenty i typy danych

Typ **int** - argumenty typu zmiennoprzecinkowego (float) zostaną skonwertowane na typ całkowitoliczbowy (int), co spowoduje utratę części ułamkowych.

```
<h3>Typy argumentów </h3>
```

```
<?php
```

```
    echo '<p>bez określania typu</p>';
```

```
    function sumNumbers($a,$b){
```

```
        return $a+$b;
```

```
    }
```

```
    $x=sumNumbers(1.5, 4.3);
```

```
    echo $x.'<br>';
```

```
    echo '<p>określony typ argumentów</p>';
```

```
    function sumNumbers2(int $a,int $b){
```

```
        return $a+$b;
```

```
    }
```

```
    $y=sumNumbers2(1.5, 4.3);
```

```
    echo $y.'<br>';// obcięcie danych do części całkowitej
```

```
    $z=sumNumbers2("3.5", 4.3);
```

```
    echo $z.'<br>';// automatyczna konwersja ciągu znaków
```

```
?>
```

Typy argumentów

bez określania typu

5.8

określony typ argumentów

5

7

PHP – tryb ścisły

Domyślnie PHP działa w trybie **nie ścisłym**.

Tryb ścisły pozwala na wymóg używania zadeklarowanych typów, wyłącza automatyczną konwersję typów danych.

Aby włączyć tryb ścisły, musisz uruchomić **każdy plik PHP (w którym chcesz, aby tryb ścisły działał)** z instrukcją:

```
<?php  
declare(strict_types=1);  
?>
```

Instrukcja musi zostać zapisana jako pierwsza deklaracja na stronie!!!

PHP – tryb ścisły

```
1 <?php
2 declare(strict_types=1);
3 // zadeklarowano ścisłe kontrolowanie typów danych
4 ?>
5 <!DOCTYPE HTML>
6 <html lang="pl">
7 <head>
8     <meta charset="utf-8"/>
9     <title>l6p16</title>
10 </head>
11 <body>
12     <h3>Typy argumentów </h3>
13     <?php
14         echo '<p>określony typ argumentów</p>';
15         function sumNumbers2(int $a,int $b){
16             return $a+$b;
17         }
18         $y=sumNumbers2(1.3, 4.5);
19         echo $y.'<br>';
20     ?>
```

Typy argumentów

określony typ argumentów

Fatal error: Uncaught TypeError: Argument 1 and defined in C:\xampp\htdocs\ZSK\l6p16.php
C:\xampp\htdocs\ZSK\l6p16.php on line 15

PHP – typ wartości zwracanej przez funkcję

Począwszy od PHP7 dopuszcza się **deklarację typu wartości zwracanej przez funkcję**. Nazwę typu należy umieścić po znaku dwukropka za nawiasem okrągłym przeznaczonym na argumenty funkcji.

Schemat takiej konstrukcji jest następujący:

```
function nazwa_funkcji():nazwa_typu{  
    // Instrukcje z wnętrza funkcji  
}
```

PHP – typ wartości zwracanej przez funkcję

<h3>Typ wartości zwracanej przez funkcję</h3>

<?php

function jakaWartosc(\$liczba):**string**{

if(\$liczba < 0){

return "ujemna";

} else if (\$liczba > 0){

return "dodatnia";

} else {

return "zero";

}

}

echo jakaWartosc(1);

?>

PHP – typ wartości zwracanej przez funkcję

Deklarując tryb ścisły znaną już instrukcją:

```
<?php  
declare(strict_types=1);  
?>
```

Można wymusić brak automatycznej konwersji typów zwracanych wartości.

PHP – typ wartości zwracanej przez funkcję

```
1 <?php
2 //declare(strict_types=1);
3 // zadeklarowano ścisłe kontrolowanie typów danych
4 ?>
5 <!DOCTYPE HTML>
6 <html lang="pl">
7 <head>
8     <meta charset="utf-8"/>
9     <title>16p18</title>
10 </head>
11 <body>
12     <h3>Typ wartości zwracanej przez funkcję</h3>
13     <?php
14         function sumNumbers($a,$b):int{
15             return $a+$b;
16         }
17         var_dump(sumNumbers(3.22,5.88));
18     ?>
```

Typ wartości zwracanej przez funkcję

int(9)

Typ wartości zwracanej przez funkcję

Fatal error: Uncaught TypeError: Return value of C:\xampp\htdocs\ZSK\l6p18.php(17): sumNumbe.

Kryteria:

6. Opisuję sposoby deklaracji typu argumentów i wartości zwracanych przez funkcję

Pokaż światłami/kciukami jak oceniasz, na ile spełniasz kryterium

PHP –funkcje-podsumowanie

Popraw kod, by działał zgodnie z oczekiwaniami:

```
<?php
    $price1 = 25;
    function calcPrice( $barrelBulk="zł", $quantity)
    {
        $result = $price1*$quantity . $barrelBulk;
    }
    calcPrice(6);
    echo $result;
?>
```

150zł